■ IT Security Audit Report

APICraft — Collaborative API Testing Platform

Audit Date: July 13, 2026
Classification: Confidential — Bachelor Thesis Internal Document
Scope: Full-Stack (Flask/Python Backend + React/JS Frontend)
Methodology: Automated Static Security Analysis

■ Critical	3
■ High	5
■ Medium	4
■ Low	2
Total	14

This report presents findings from a comprehensive static security analysis of the APICraft codebase. 14 distinct vulnerabilities were identified across authentication, authorization, network security, data exposure, input validation, and configuration hardening domains.

Executive Summary

ID	Vulnerability	Category	Severity	OWASP
SEC-01	Hardcoded JWT Secret Key	Authentication	Critical	A02: Cryptographic Failures
SEC-02	Unrestricted Server-Side Request Forgery (SSRF)	Injection / Access Control	Critical	A10: Server-Side Request Forgery
SEC-03	Wildcard CORS — All Origins Allowed	Network Security	Critical	A05: Security Misconfiguration
SEC-04	JWT Token Stored in sessionStorage (XSS-Accessible)	Authentication	High	A02: Cryptographic Failures
SEC-05	No Rate Limiting on Authentication Endpoints	Authentication	High	A07: Identification & Authentication Failures
SEC-06	Flask Debug Mode Enabled in Production Entry Point	Configuration	High	A05: Security Misconfiguration
SEC-07	Sensitive Credentials Stored Unencrypted in History	Data Exposure	High	A02: Cryptographic Failures
SEC-08	Admin Role Determined by Client-Side sessionStorage Flag	Authorization	High	A01: Broken Access Control
SEC-09	GET /collections Has No Ownership Check (IDOR)	Authorization	Medium	A01: Broken Access Control
SEC-10	No Input Length Validation on User-Supplied Fields	Input Validation	Medium	A03: Injection
SEC-11	Hardcoded Database Connection String	Configuration	Medium	A05: Security Misconfiguration
SEC-12	Custom JWT Implementation Instead of Proven Library	Cryptography	Medium	A02: Cryptographic Failures
SEC-13	Missing HTTP Security Headers	Network Security	Low	A05: Security Misconfiguration
SEC-14	Audit Log Cascade Delete Destroys Forensic Evidence	Data Integrity	Low	A09: Security Logging and Monitoring Failures

Detailed Findings

SEC-01	Hardcoded JWT Secret Key	■ Critical

Category:	Authentication	**OWASP:**	A02: Cryptographic Failures
File(s):	`backend/services/jwt_service.py, Line 10`		

Description

The JWT secret key falls back to a hardcoded literal string 'apicraft-jwt-development-secret-key-38491024' when the JWT_SECRET environment variable is not set. In any environment where JWT_SECRET is not explicitly configured, this default value is active.

Risk

Any attacker with access to the source code can forge valid JWT tokens for any user ID — including admin accounts — without needing credentials. A forged token passes all @token_required checks successfully.

Impact

Complete authentication bypass. Full impersonation of any user or admin.

Vulnerable Code

```
SECRET_KEY = os.environ.get('JWT_SECRET', 'apicraft-jwt-development-secret-key-38491024')
```

Recommended Fix

Remove the default fallback entirely. Raise a RuntimeError at startup if JWT_SECRET is not set. Generate a random 256-bit key: python -c "import secrets; print(secrets.token_hex(32))"

SEC-02	Unrestricted Server-Side Request Forgery (SSRF)	■ Critical

Category:	Injection / Access Control	**OWASP:**	A10: Server-Side Request Forgery
File(s):	`backend/routes/api_client_routes.py + backend/services/api_client_service.py`		

Description

The /api/execute endpoint accepts an arbitrary URL from any authenticated user and makes an outbound HTTP request from the server. There is zero validation on the destination URL, protocol, or resolved IP address.

Risk

An authenticated user can direct the backend to request internal network resources (e.g., http://localhost:5000/api/admin/users), cloud metadata endpoints (e.g., http://169.254.169.254/latest/meta-data/ on AWS which exposes IAM credentials), internal database ports, or other microservices not exposed to the public internet.

Impact

Internal network reconnaissance, cloud credential theft, data exfiltration, infrastructure pivoting.

Vulnerable Code

```
result = execute_request(method.upper(), url, params, headers, body) # No URL validation
```

Recommended Fix

Validate the URL resolves to a public IP before making the request. Block all private IP ranges: 10.x.x.x, 172.16-31.x.x, 192.168.x.x, 127.x.x.x, 169.254.x.x. Consider an allowlist of permitted domains.

SEC-03	Wildcard CORS — All Origins Allowed	■ Critical

Category:	Network Security	**OWASP:**	A05: Security Misconfiguration
File(s):	`backend/app.py, Line 36`		

Description

CORS(app) is called with no configuration, enabling cross-origin requests from any domain (Access-Control-Allow-Origin: *). Any website on the internet can make authenticated cross-origin requests to the API.

Risk

Enables CSRF-like attacks where malicious third-party sites can read API responses. Combined with sessionStorage token storage, an XSS payload can exfiltrate tokens to any external origin. Responses containing sensitive data can be read by any origin.

Impact

Data theft, unauthorized API calls triggered from malicious third-party sites.

Vulnerable Code

```
CORS(app) # Wildcard - accepts requests from any origin
```

Recommended Fix

Restrict to known frontend origin: CORS(app, origins=['http://localhost:3000'], supports_credentials=True). In production, read from FRONTEND_ORIGIN environment variable.

SEC-04	JWT Token Stored in sessionStorage (XSS-Accessible)	■ High

Category:	Authentication	**OWASP:**	A02: Cryptographic Failures
File(s):	`frontend/src/contexts/AuthContext.js + frontend/src/services/api.js`		

Description

The JWT bearer token is stored in browser sessionStorage and read on every API request. sessionStorage is fully accessible to any JavaScript code running on the page — including injected XSS payloads.

Risk

If a Cross-Site Scripting (XSS) vulnerability exists anywhere in the React app (e.g., via an unsanitized user comment rendered as HTML), an attacker can steal the JWT token with a single line of JavaScript and use it from any machine until the token expires (24 hours).

Impact

Full account takeover for any user whose token is stolen.

Vulnerable Code

```
token: sessionStorage.getItem('token'),
headers['Authorization'] = `Bearer ${token}`;
```

Recommended Fix

Use httpOnly cookies which are inaccessible to JavaScript. On login, the backend sets a httpOnly, Secure, SameSite=Strict cookie. The frontend stops reading/writing sessionStorage for the token.

SEC-05	No Rate Limiting on Authentication Endpoints	■ High

Category:	Authentication	**OWASP:**	A07: Identification & Authentication Failures
File(s):	`backend/routes/auth_routes.py`		

Description

The /api/auth/login and /api/auth/signup endpoints have no rate limiting. An attacker can make unlimited requests per second to these endpoints.

Risk

Enables credential stuffing attacks (testing large lists of stolen credentials), brute force attacks against user passwords, and resource exhaustion via the CPU-intensive bcrypt hashing (12 rounds per attempt) which can be triggered with unlimited concurrent requests.

Impact

Account takeover via brute force; CPU exhaustion denial of service.

Vulnerable Code

```
@auth_bp.route('/api/auth/login', methods=['POST'])
def login():
# No rate limiting applied
```

Recommended Fix

Install Flask-Limiter and apply: @limiter.limit('10 per minute') on /login and @limiter.limit('5 per minute') on /signup. Consider progressive delays or CAPTCHA after repeated failures.

SEC-06	Flask Debug Mode Enabled in Production Entry Point	■ High

Category:	Configuration	**OWASP:**	A05: Security Misconfiguration
File(s):	`backend/app.py, Line 122`		

Description

The application is started unconditionally with debug=True. Flask's debug mode enables the Werkzeug interactive debugger which renders an interactive Python debugger page in the browser when an exception occurs.

Risk

The interactive debugger allows arbitrary Python code execution in the browser. Any unhandled exception triggered by a malformed request gives an attacker remote code execution on the server. Additionally, full stack traces with local variable values (including secrets) are exposed in HTTP error responses.

Impact

Remote code execution on the server; complete server compromise.

Vulnerable Code

```
if __name__ == '__main__':
app = create_app()
app.run(debug=True) # Always enabled!
```

Recommended Fix

Read debug flag from environment: debug_mode = os.environ.get('FLASK_DEBUG', 'false').lower() == 'true'. Never set FLASK_DEBUG=true in production. Use Gunicorn or uWSGI as the production WSGI server.

SEC-07	Sensitive Credentials Stored Unencrypted in History	■ High

Category:	Data Exposure	**OWASP:**	A02: Cryptographic Failures
File(s):	`backend/models/history_model.py + backend/services/history_service.py`		

Description

Every API request executed through APICraft is saved to the history log including headers, auth, params, and body as plain JSON. API keys, Bearer tokens, Basic Auth credentials, and any secrets passed as headers or body are stored in plaintext.

Risk

A database breach exposes all users' external API credentials. Admins can view other users' credentials via the admin_service workspace inspection feature. History data returned to the frontend is logged by browser developer tools and proxies.

Impact

Mass credential exposure for all external services tested through APICraft.

Vulnerable Code

```
headers = db.Column(db.JSON, nullable=True) # Stores: Authorization: Bearer
auth = db.Column(db.JSON, nullable=True) # Stores: API keys, passwords
```

Recommended Fix

Encrypt sensitive fields at rest using cryptography.fernet with a key from environment. Mask sensitive header values in API responses (e.g., Bearer ****). Consider giving users an explicit opt-out from saving sensitive data.

SEC-08	Admin Role Determined by Client-Side sessionStorage Flag	■ High

Category:	Authorization	**OWASP:**	A01: Broken Access Control
File(s):	`frontend/src/contexts/AuthContext.js + frontend/src/App.js`		

Description

The isAdmin flag is read from sessionStorage on every page load. A regular user can open browser DevTools and set sessionStorage.setItem('isAdmin', 'true'), causing the frontend to render and navigate to the /admin route.

Risk

Although the backend re-validates admin status for each API call, the frontend renders the admin dashboard UI and dispatches admin API requests based solely on the client-side flag. This exposes admin UI components and makes admin API calls that are then rejected — but the UI state is already compromised.

Impact

Unauthorized access to admin UI; confusion between client/server authorization state.

Vulnerable Code

```
isAdmin: sessionStorage.getItem('isAdmin') === 'true', // Untrusted client storage
```

Recommended Fix

Do not use sessionStorage for authorization decisions. Derive admin status from a protected API call on page load. Include is_admin as a verified JWT claim and add an @admin_required decorator that validates the JWT claim directly.

SEC-09	GET /collections Has No Ownership Check (IDOR)	■ Medium

Category:	Authorization	**OWASP:**	A01: Broken Access Control
File(s):	`backend/routes/collection_routes.py, Lines 13–16`		

Description

The list_collections endpoint fetches all collections for a workspace without verifying that the requesting user is a member or owner of that workspace. Any authenticated user can read any workspace's collections by iterating IDs.

Risk

Insecure Direct Object Reference (IDOR): any authenticated user can enumerate collections of all workspaces (GET /api/workspaces/1/collections, /2/collections, etc.). Exposes collection names, request names, and API endpoint structures of other users.

Impact

Unauthorized data disclosure; enumeration of other users' API collections.

Vulnerable Code

```
def list_collections(workspace_id):
return jsonify(get_collections_by_workspace(workspace_id)), 200 # g.user_id not checked
```

Recommended Fix

Pass g.user_id to get_collections_by_workspace and enforce the same ownership/membership check used in all other workspace endpoints.

SEC-10	No Input Length Validation on User-Supplied Fields	■ Medium

Category:	Input Validation	**OWASP:**	A03: Injection
File(s):	`backend/routes/auth_routes.py, comment_routes.py, request_routes.py`		

Description

No maximum length constraints are enforced on user-supplied text inputs. Fields like username, password, email, comment content, workspace name, request name, and URL accept unbounded strings from any request.

Risk

Submitting a very long password (e.g., 100,000 chars) forces the server to bcrypt-hash it with 12 rounds, exhausting CPU. Database NVARCHAR(MAX) fields accept unlimited data, enabling storage-based DoS. Note: bcrypt truncates at 72 bytes, so passwords over 72 chars may silently match shorter versions.

Impact

CPU exhaustion denial of service; storage-based DoS; bcrypt truncation bugs.

Vulnerable Code

```
password = data.get('password') # No length check before bcrypt.hashpw(...)
```

Recommended Fix

Add explicit length checks: if len(password) > 128: return error. Use a validation library like marshmallow or pydantic for structured validation.

SEC-11	Hardcoded Database Connection String	■ Medium

Category:	Configuration	**OWASP:**	A05: Security Misconfiguration
File(s):	`backend/app.py, Lines 31–34`		

Description

The database connection string including server hostname (MSI\SQLEXPRESS01) and database name (API_tester) is hardcoded directly in application source code, committed to version control.

Risk

Infrastructure details are exposed to anyone with source code access. Changing environments requires code changes. If credentials are ever added to the connection string (SQL Server auth instead of Windows auth), they would be permanently stored in git history.

Impact

Infrastructure disclosure; credential leak risk; deployment inflexibility.

Vulnerable Code

```
app.config['SQLALCHEMY_DATABASE_URI'] = "mssql+pyodbc://@MSI\\SQLEXPRESS01/API_tester?..."
```

Recommended Fix

Read from environment variable: DATABASE_URL = os.environ.get('DATABASE_URL'). Raise RuntimeError if not set. Never commit connection strings to version control.

SEC-12	Custom JWT Implementation Instead of Proven Library	■ Medium

Category:	Cryptography	**OWASP:**	A02: Cryptographic Failures
File(s):	`backend/services/jwt_service.py`		

Description

The JWT implementation is hand-written using Python's hmac, hashlib, base64, and json modules rather than using a well-audited library. The JWT header's algorithm field is not validated during token decoding.

Risk

Hand-rolled cryptographic code is prone to subtle bugs not present in audited libraries. The alg field is ignored during decoding — a future change could introduce algorithm confusion attacks. No token revocation mechanism exists: suspended users' tokens remain valid for up to 24 hours after suspension.

Impact

Potential cryptographic bugs; no token revocation on account changes.

Vulnerable Code

```
# Custom HMAC-SHA256 JWT without library
hmac.new(SECRET_KEY.encode('utf-8'), signing_input, hashlib.sha256).digest()
```

Recommended Fix

Replace with PyJWT (pip install PyJWT). It is battle-tested, handles all edge cases, and is regularly audited. Implement a token blacklist table for revocation.

SEC-13	Missing HTTP Security Headers	Low

Category:	Network Security	**OWASP:**	A05: Security Misconfiguration
File(s):	`backend/app.py (global middleware)`		

Description

The Flask backend does not set any standard HTTP security headers: no Content-Security-Policy, no X-Content-Type-Options, no X-Frame-Options, no Referrer-Policy, no Permissions-Policy, and no Strict-Transport-Security.

Risk

Without CSP, XSS payloads can load scripts from arbitrary external origins. Without X-Frame-Options, the app can be embedded in iframes for clickjacking. Without X-Content-Type-Options, browsers may MIME-sniff responses.

Impact

Increased XSS attack surface; clickjacking vulnerability; MIME-type confusion.

Vulnerable Code

```
# No security headers set anywhere in app.py or middleware
```

Recommended Fix

Add @app.after_request hook: set X-Content-Type-Options: nosniff, X-Frame-Options: DENY, Referrer-Policy, and Content-Security-Policy headers.

SEC-14	Audit Log Cascade Delete Destroys Forensic Evidence	Low

Category:	Data Integrity	**OWASP:**	A09: Security Logging and Monitoring Failures
File(s):	`backend/models/audit_log_model.py, Line 8`		

Description

The admin_audit_log table uses ondelete='CASCADE' on the FK to users. When an admin user is deleted, all their audit log entries are permanently deleted, destroying the forensic record of all their past administrative actions.

Risk

A malicious admin who deletes their own account (or has it deleted) erases all evidence of their past actions: user suspensions, deletions, promotions, and workspace deletions. This undermines the entire purpose of the audit log.

Impact

Loss of forensic evidence; audit trail manipulation; compliance failure.

Vulnerable Code

```
admin_id = db.Column(db.Integer, db.ForeignKey('users.id', ondelete='CASCADE'), nullable=False)
```

Recommended Fix

Change to ondelete='SET NULL' and make admin_id nullable=True. Audit entries survive admin account deletion, preserving the forensic record.

OWASP Top 10 Coverage

OWASP Category	Vulnerabilities Found
A01: Broken Access Control	SEC-08, SEC-09
A02: Cryptographic Failures	SEC-01, SEC-04, SEC-07, SEC-12
A03: Injection	SEC-10
A05: Security Misconfiguration	SEC-03, SEC-06, SEC-11, SEC-13
A07: Identification & Auth Failures	SEC-05
A09: Security Logging Failures	SEC-14
A10: Server-Side Request Forgery	SEC-02

Prioritized Remediation Plan

Phase 1 — Immediate (Fix Within 24 Hours)

ID	Action Required	Effort
SEC-01	Remove hardcoded JWT secret; require env var	~30 min
SEC-03	Restrict CORS to frontend origin only	~15 min
SEC-06	Disable unconditional debug=True in app.run()	~10 min

Phase 2 — Short-Term (Fix Within 1 Week)

ID	Action Required	Effort
SEC-02	Add SSRF URL validation with private IP blocklist	~2 h
SEC-04	Migrate JWT token storage to httpOnly cookie	~4 h
SEC-05	Add Flask-Limiter rate limiting to auth endpoints	~1 h
SEC-08	Fix admin authorization: use JWT claim, not sessionStorage	~2 h
SEC-11	Move DB URI to DATABASE_URL environment variable	~30 min

Phase 3 — Medium-Term (Fix Within 1 Month)

ID	Action Required	Effort
SEC-07	Encrypt sensitive history fields (headers, auth, body) at rest	~1 day
SEC-09	Add ownership check to GET /workspaces/{id}/collections	~30 min
SEC-10	Add input length validation on all user-supplied fields	~2 h
SEC-12	Replace custom JWT implementation with PyJWT library	~3 h

Phase 4 — Hardening (Before Production Release)

ID	Action Required	Effort
SEC-13	Add HTTP security headers via @app.after_request hook	~30 min
SEC-14	Fix audit log FK to SET NULL instead of CASCADE	~30 min

This report was generated by automated static code analysis. All findings should be validated by a human security engineer before remediation planning is finalized. Findings reflect the state of the codebase at the time of the audit.